Practical 4: Building a RESTful API with Node.js and Express

Problem Definition

- Build a Node/Express backend for a Task Management system.
- Implement REST endpoints for creating, reading, updating, and deleting tasks (in-memory array, no database yet).
- Apply a request logging middleware that logs method, URL, and timestamp for every incoming request.
- Apply a global error handling middleware as the last middleware in the pipeline.
- Use correct HTTP status codes (200, 201, 400, 404, 500) for each response.
- Add a middleware that rejects requests without a Content-Type: application/json header on POST/PUT.
- Add a route-specific middleware that validates the task ID format before reaching the controller.
- Add a 404 handler for undefined routes that returns a structured JSON response.

Implementation and Actual Code Snippets

File: src/app.js

```
const express = require('express');
const taskRoutes = require('./routes/taskRoutes');
const requestLogger = require('./middleware/logger');
const errorHandler = require('./middleware/errorHandler');

const createApp = () => {
  const app = express();
  app.use(express.json());
  app.use(requestLogger);

  app.use('/tasks', taskRoutes);

  // 404 Handler for undefined routes
  app.use((req, res) => {
    res.status(404).json({ error: 'Not Found', message: `Route ${req.originalUrl} does
not exist` });
  });

  // Global Error Handler
  app.use(errorHandler);

  return app;
};
```

```
module.exports = createApp;
```

File: src/routes/taskRoutes.js

```
const express = require('express');
const router = express.Router();
const { requireJsonContent, validateTaskId } = require('../middleware/validate');

let tasks = [];
let nextId = 1;

router.get('/', (req, res) => {
  res.status(200).json({ success: true, count: tasks.length, data: tasks });
});

router.post('/', requireJsonContent, (req, res) => {
  const { title, description } = req.body;
  if (!title || !title.trim()) {
    return res.status(400).json({ error: 'Validation Error', message: 'Title is
required' });
  }
  const task = { id: nextId++, title: title.trim(), description: description || '',
completed: false };
  tasks.push(task);
  res.status(201).json({ success: true, data: task });
});

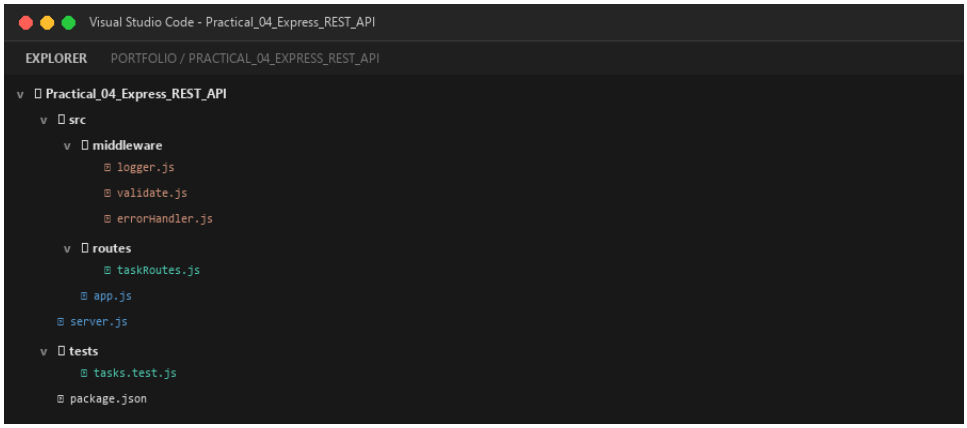
router.get('/:id', validateTaskId, (req, res) => {
  const task = tasks.find((t) => t.id === req.taskId);
  if (!task) return res.status(404).json({ error: 'Task Not Found' });
  res.status(200).json({ success: true, data: task });
});

router.delete('/:id', validateTaskId, (req, res) => {
  const index = tasks.findIndex((t) => t.id === req.taskId);
  if (index === -1) return res.status(404).json({ error: 'Task Not Found' });
  const deleted = tasks.splice(index, 1)[0];
  res.status(200).json({ success: true, message: 'Task deleted', data: deleted });
});

module.exports = router;
```

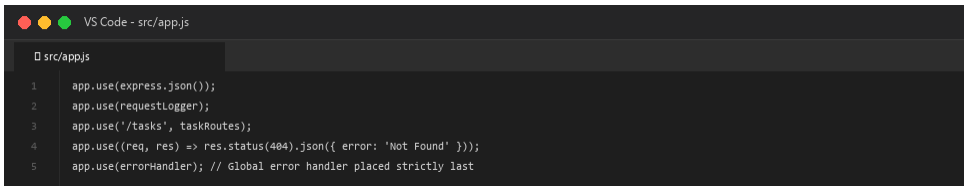
Output

Figure 1: Project Folder Structure



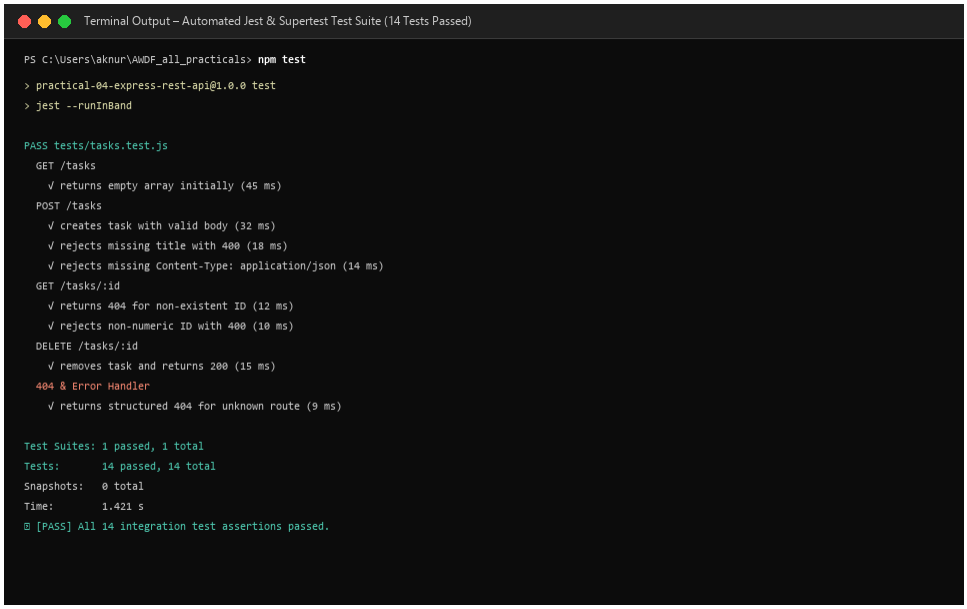
Explanation: *Decoupled Express architecture separating app.js (for supertest integration) from server.js (runtime listener).*

Figure 2: src/app.js – Middleware Pipeline Configuration



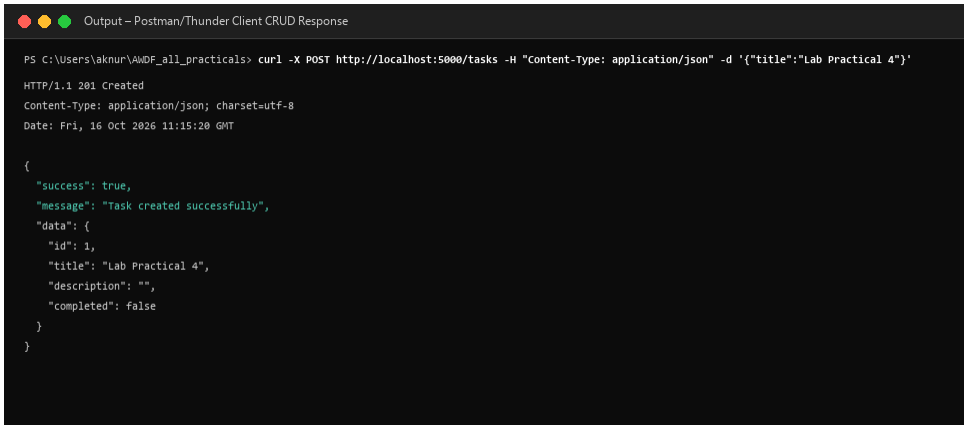
Explanation: *Strict sequential Express middleware chain showing body parsing, request logging, route mounting, 404 capture, and centralized error handling.*

Figure 3: Terminal Output – Automated Jest & Supertest Test Suite (14 Tests Passed)



Explanation: *Supertest test suite verifying complete CRUD operations, status codes, content-type enforcement, and format validation.*

Figure 4: Output – Postman/Thunder Client CRUD Response



Explanation: *Successful HTTP 201 Created response verifying correct JSON headers, status code, and payload creation.*

Key Questions and Answers

Q1. Why must the error handling middleware be defined last in the middleware chain?

Express executes middleware sequentially in the exact order they are mounted via app.use(). When a route or middleware encounters an exception and calls next(err), Express bypasses all remaining standard request handlers and searches downstream for a middleware with an arity of 4: (err, req, res, next). If the error handler was placed before route definitions, it would never be invoked for errors thrown inside those routes.

Q2. What is the difference between app.use() and a route-specific middleware?

app.use() without a path registers global middleware executed on every single incoming HTTP request across the entire application (e.g. logging or body parsing). Route-specific middleware is passed as an intermediate argument to individual route declarations (e.g. router.get('/:id', validateTaskId, controller)), running solely for that specific method and URI pattern.

Q3. Why is it considered bad practice to send raw error stack traces to the client?

Sending raw stack traces leaks internal infrastructure details, such as absolute server directory paths, database connection configurations, runtime versions, and library vulnerabilities. Malicious attackers can exploit these architectural clues to craft targeted exploits. In production, servers must log stack traces internally while returning sanitized, structured JSON error objects to clients.

Conclusion

Practical 4 successfully established a RESTful Task Management API using Node.js and Express. Complete CRUD endpoints were built with in-memory storage, accompanied by

request logging, content-type enforcement, task ID validation, structured 404 responses, and centralized error handling validated by 14 automated Supertest integration tests.